Prac-04

To create a Python Application

```
PS C:\Users\narut> mkdir docker_python


    Directory: C:\Users\narut


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        17-02-2026     12:31                docker_python


PS C:\Users\narut> cd docker_python
PS C:\Users\narut\docker_python> notepad app.py
PS C:\Users\narut\docker_python> cat app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Docker!"
```

Step1: create a folder

Step2: create a file name app.py

from flask import Flask

app = Flask(_name_)

@app.route("/")

def home():

return "Hello from Docker!”

app.run(host="0.0.0.0", port=5000)

```
PS C:\Users\narut\docker_python> cat requirement.txt
Flask
PS C:\Users\narut\docker_python> notepad dockerfile
PS C:\Users\narut\docker_python> cat dockerfile
```

Step3: create another filename requirement.txt (write flask in it)

Step4: create the dockerfile

1. Base image

FROM python:3.11-slim

2. Set working directory

WORKDIR /app

3. Copy dependency file

COPY requirements.txt

.# 4. Install dependencies

RUN pip install --no-cache-dir -r requirements.txt

5. Copy application code

COPY . .

6. Expose port

EXPOSE 5000

7. Run the app

CMD ["python", "app.py"]

```
PS C:\Users\narut\docker_python> ls


    Directory: C:\Users\narut\docker_python


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        17-02-2026     12:39            185 app.py
-a----        17-02-2026     12:44            349 dockerfile
-a----        17-02-2026     12:43              5 requirement.txt
```

Step5: check the directory

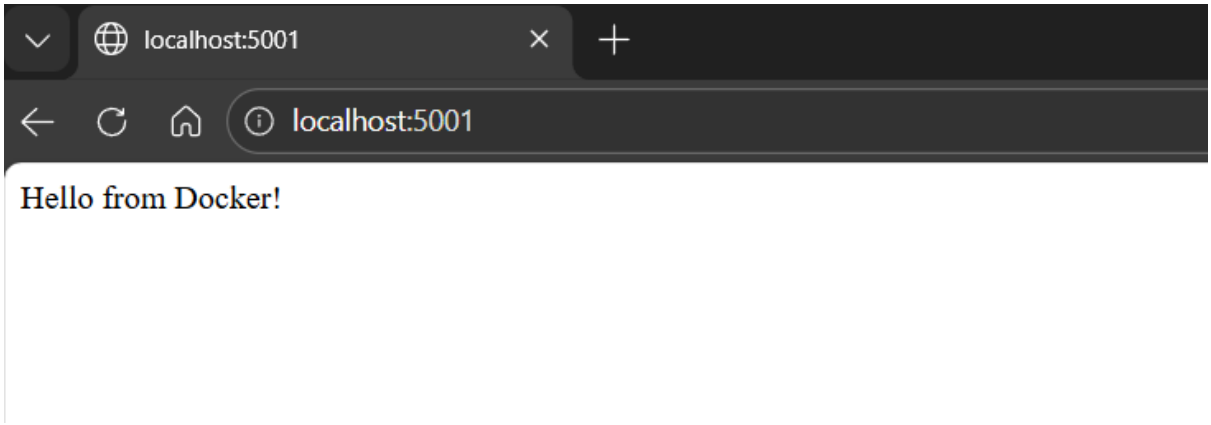
```
Terminal

PS C:\Users\narut\docker_python> docker build -t python11 .
[+] Building 415.1s (12/12) FINISHED                                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                                   0.0s
 => => transferring dockerfile: 377B                                                                                   0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                    5.2s
 => [auth] library/python:pull token for registry-1.docker.io                                                          0.0s
 => [internal] load .dockerignore                                                                                      0.0s
 => => transferring context: 2B                                                                                        0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:0b23cfb7425d065008b778022a17b1551c82f8b4866ee5a7a200084b7e2eafbf  398.8s
 => => resolve docker.io/library/python:3.11-slim@sha256:0b23cfb7425d065008b778022a17b1551c82f8b4866ee5a7a200084b7e2eafbf   2.6s
 => => sha256:d85099f0969e8b2306770a12dffcda300208cc3a18b876c0ad3dc0cb51aefb9b 250B / 250B                             1.4s
 => => sha256:8cbc47ff628d718fb76f7fca9897e4e8b607a4f543008cdee760705eecea1b24 14.36MB / 14.36MB                      131.9s
 => => sha256:64faa99400e1388ed9f202917ada9fac34fd46c950d40cd4102364ee9d6ab804 1.29MB / 1.29MB                         30.9s
 => => sha256:0c8d55a45c0dc58de60579b9cc5b708de9e7957f4591fc7de941b67c7e245da0 29.78MB / 29.78MB                      393.3s
 => => extracting sha256:0c8d55a45c0dc58de60579b9cc5b708de9e7957f4591fc7de941b67c7e245da0                               1.4s
 => => extracting sha256:64faa99400e1388ed9f202917ada9fac34fd46c950d40cd4102364ee9d6ab804                               0.2s
 => => extracting sha256:8cbc47ff628d718fb76f7fca9897e4e8b607a4f543008cdee760705eecea1b24                               1.0s
 => => extracting sha256:d85099f0969e8b2306770a12dffcda300208cc3a18b876c0ad3dc0cb51aefb9b                               0.0s
 => [internal] load build context                                                                                      0.0s
 => => transferring context: 108B                                                                                      0.0s
 => [auth] library/python:pull token for registry-1.docker.io                                                          0.0s
```

Step6: Build the image

```
 => => naming to docker.io/library/python11:latest                                                                     0.0s
 => => unpacking to docker.io/library/python11:latest                                                                  0.6s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ommq5yz660owghp1gatevwckr
PS C:\Users\narut\docker_python> docker images
                                                                                                  i Info →  U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest                        25109184c71b       13.1MB         3.95MB    U
ashika0804/ab2:v1.0                  28e750845649        476MB          148MB
PS C:\Users\narut\docker_python> docker run -p 5001:5000 python11
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [18/Feb/2026 06:49:06] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [18/Feb/2026 06:49:06] "GET /favicon.ico HTTP/1.1" 404 -
```

Step7: check the image build

Step8: do the port mapping



Step9: run the server on the browser

Create another application from python using flask

```
PS C:\Users\narut\docker_python> notepad app2.py
PS C:\Users\narut\docker_python> ren dockerfile dockerfile.txt
```

Step1: Create the new app2.py file

Step2: Write code

```
from flask import Flask, request


app = Flask(__name__)


@app.route("/", methods=["GET", "POST"])
def home():
    if request.method == "POST":
        name = request.form.get("name")
        return f"Hello {name}! 👋 (from Docker)"

    return """
        <h2>Simple Flask Form</h2>
        <form method="post">
            <input type="text" name="name" placeholder="Enter name" required>
            <button type="submit">Submit</button>
        </form>
    """
```

```
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step3: Again, rename the dockerfile to dockerfile.txt

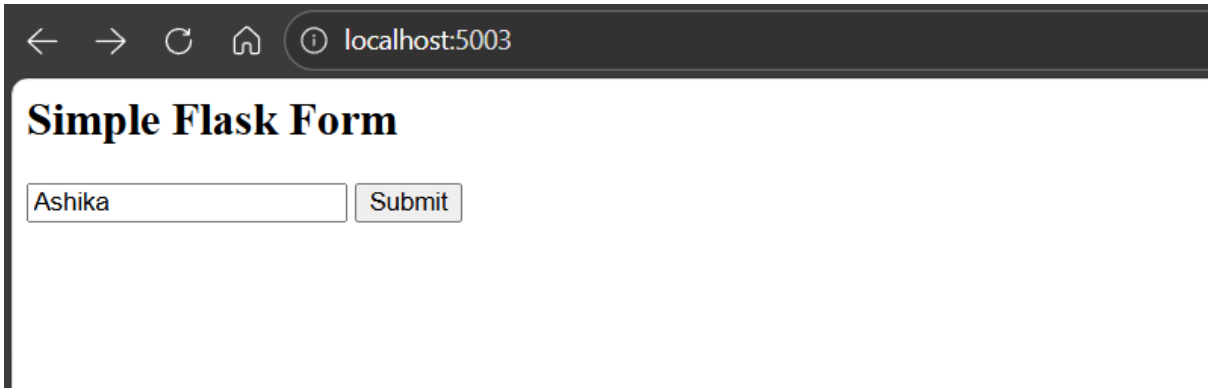
Step4: Make the changes in that dockerfile.txt

```
# 1. Base image
FROM python:3.11-slim

# 2. Set working directory
WORKDIR /app

# 3. Copy dependency file
COPY requirements.txt

.# 4. Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy application code
COPY . .

# 6. Expose port
EXPOSE 5000

# 7. Run the app
CMD ["python", "app2.py"]
```

Step5: Save the file and exit then rename the file from .txt to dockerfile

```
Terminal
PS C:\Users\narut\docker_python> docker build -t python2 .
[+] Building 6.6s (11/11) FINISHED                                                    docker:desktop-linux
 => [internal] load build definition from dockerfile                                                  0.0s
 => => transferring dockerfile: 378B                                                                  0.0s
 => [internal] load metadata for docker.io/library/python:3.11-slim                                   6.3s
 => [auth] library/python:pull token for registry-1.docker.io                                         0.0s
 => [internal] load .dockerignore                                                                     0.0s
 => => transferring context: 2B                                                                       0.0s
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:0b23cfb7425d065008b778022a17b1551c82f8b4866ee5a7a200084b7e2eafbf   0.0s
 => => resolve docker.io/library/python:3.11-slim@sha256:0b23cfb7425d065008b778022a17b1551c82f8b4866ee5a7a200084b7e2eafbf   0.0s
 => [internal] load build context                                                                     0.0s
 => => transferring context: 1.03kB                                                                   0.0s
 => CACHED [2/5] WORKDIR /app                                                                         0.0s
 => CACHED [3/5] COPY requirements.txt .                                                              0.0s
```

Step6: Build the image and do the port mapping

```
Terminal

 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
172.17.0.1 - - [18/Feb/2026 07:06:34] "GET / HTTP/1.1" 200 -
172.17.0.1 - - [18/Feb/2026 07:06:34] "GET /favicon.ico HTTP/1.1" 404 -
172.17.0.1 - - [18/Feb/2026 07:11:33] "POST / HTTP/1.1" 200 -
172.17.0.1 - - [18/Feb/2026 07:11:37] "GET / HTTP/1.1" 200 -
```

Step7: Run the server on browser